

Muhammad Awais
21I-1377
BS(DS)-8B

TECHNICAL REPORT: TITANIC SURVIVAL PREDICTION ORCHESTRATED BY APACHE AIRFLOW AND TRACKED BY MLFLOW

1. Architecture Explanation (Airflow + MLflow Interaction):

System Overview:

This pipeline implements an end-to-end Machine Learning Operations (MLOps) workflow, using two complementary tools i.e. Apache Airflow and MLFlow.

Apache Airflow:

Serves as the workflow orchestrator, defining task dependencies, managing execution order, handling retries, and enabling parallel processing through a Directed Acyclic Graph aka DAG.

MLFlow:

Serves as the experiment tracking and model management platform, logging hyperparameters, metrics, artifacts of models, and maintaining a model registry for approved models.

Interaction Pattern:

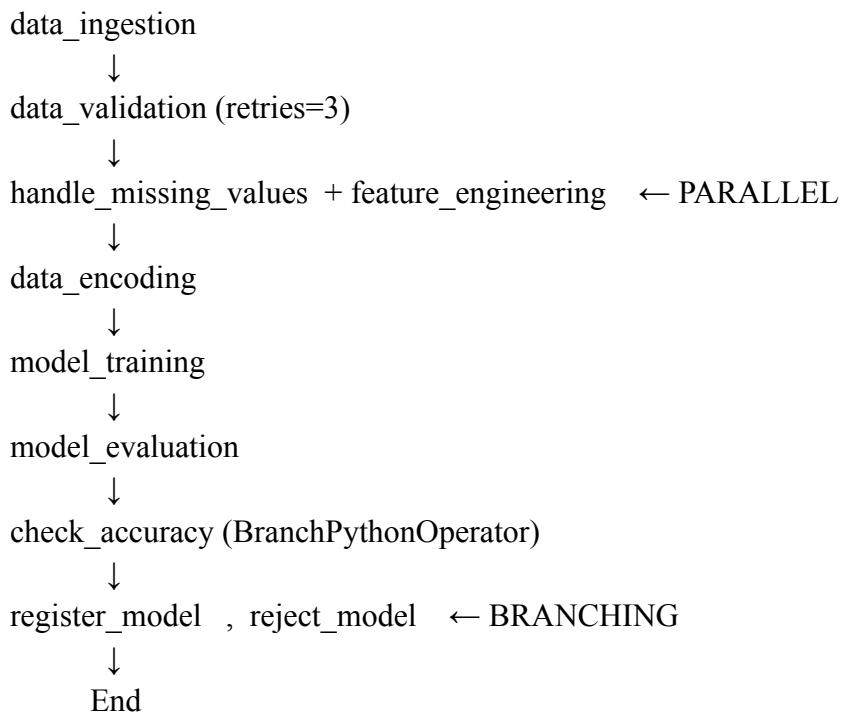
1. Airflow triggers each pipeline stage as a Python task.
2. During the **Model Training** task, an MLflow run is initiated. The model type, hyperparameters, training dataset size, and the trained model artifact are logged.
3. During **Model Evaluation**, metrics (accuracy, precision, recall, F1-score) are logged to the same MLflow run.
4. The **Branching Logic** uses Airflow's BranchPythonOperator to decide whether to register the model in MLflow's Model Registry if accuracy ≥ 0.80 or reject it by giving logging the rejection reason.
5. Data is passed between tasks using Airflow's XCom mechanism e.g., file paths, accuracy values, MLflow run IDs

Data Flow:

CSV File → Ingestion → Validation → [Missing Values | Feature Engineering]
→ Encoding → Training (MLflow logs) → Evaluation (MLflow metrics)
→ Branch Decision → Register/Reject (MLflow Registry)

2. DAG Structure and Dependency Explanation:

Task Dependencies (Linear + Parallel + Branching):



Dependency Explanation:

Sequential:

Ingestion must complete before validation, which must pass before processing begins.

Parallel:

handle_missing_values and **feature_engineering** run simultaneously as they operate independently on the raw data.

Convergence:

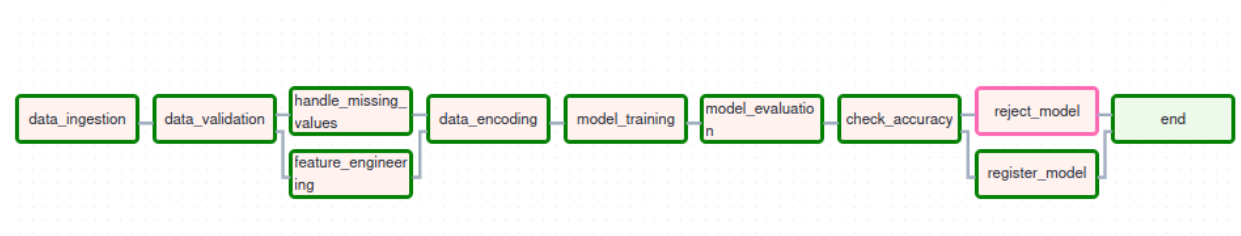
data_encoding waits for BOTH parallel tasks to complete.

Branching:

check_accuracy returns either **register_model** or **reject_model** as the next task ID, and Airflow only executes the chosen branch.

No Cycles:

The graph is strictly acyclic every edge points forward in the pipeline.



3. Experiment Comparison Analysis:

Experiment Results Summary

Run	Model	Key HyperParams	Accuracy	Precision	Recall	F1-Score	Registered?
1	Logistic Regression	C=1.0, max_iter=200	0.80	0.80	0.80	0.80	Yes
2	Random Forest	n_estimators=100, max_depth=10	0.83	0.83	0.83	0.83	Yes
3	Randome forest	n_estimators=200, max_depth=5	0.81	0.81	0.81	0.81	Yes

Analysis:

Best Performing Model:

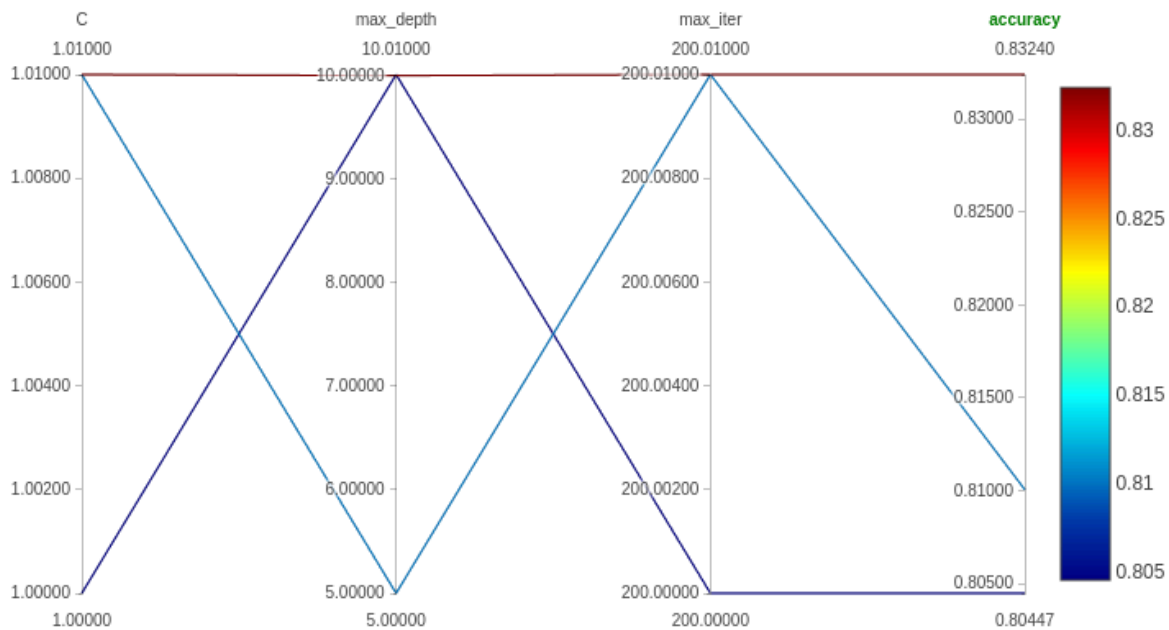
Run 2 (Random Forest, n_estimators=100, max_depth=10) achieved the highest accuracy and F1-score. The deeper trees (max_depth=10) captured more complex feature interactions in the Titanic dataset.

Logistic Regression:

Provided a strong baseline but was limited by its linear decision boundary.

Run 3:

With shallower trees (max_depth=5) showed slight underfitting, suggesting that some important feature interactions require deeper splits.



4. Failure and Retry Explanation:

Retry Configuration:

The data_validation task is configured with:

- retries=3: Maximum 3 retry attempts after initial failure
- retry_delay=timedelta(seconds=5): 5-second wait between retries

Demonstration:

To demonstrate retry behavior, the Age column was intentionally corrupted to have:

50% missing values (exceeding the 30% threshold).

The validation task:

1. **Attempt 1:** Raised value error. Age missing percentage exceeded 30%.
2. **Attempt 2:** Same failure (data unchanged between retries).
3. **Attempt 3:** Same failure.
4. **Final Status:** Task marked as FAILED after exhausting all retries.

This demonstrates Airflow's built-in fault tolerance mechanism. In production, transient failures e.g., network timeouts when loading data would be automatically retried.

Evidence:

(by attempts)

1 2 3 4

All Levels All File Sources ☐ Wrap Download See More

```
muhammad-awais-Latitude-E5470
*** Found local files:
***   * /home/muhammad-awais/mlops_assignment/airflow/logs/dag_id=mlops_airflow_mlflow_pipeline/run_id=manual__2026-03-11T14:57:34.725893+00:00/task
[2026-03-11, 14:58:18 UTC] {local_task_job_runner.py:120} ▶ Pre task execution logs
[2026-03-11, 14:58:18 UTC] {logging_mixin.py:188} INFO - Age missing: 58.81%
[2026-03-11, 14:58:18 UTC] {logging_mixin.py:188} INFO - Embarked missing: 0.22%
[2026-03-11, 14:58:18 UTC] {taskinstance.py:441} ▶ Post task execution logs
```

5. Reflection Questions on Production Deployment:

Q1: How would you deploy this pipeline in production?

I would run Airflow and MLflow as separate services ideally with Docker/Kubernetes instead of on a laptop. Airflow would use PostgreSQL for its metadata DB, and MLflow would use a database backend plus shared artifact storage like AWS S3. I'd also keep separate dev/test/prod environments.

Q2: What monitoring would you add?

For Airflow I will add alerts on failures and task-duration monitoring. For ML I will fo for basic data-drift checks and tracking model performance after deployment, so we notice accuracy drop over time.

Q3: How would you handle data versioning?

I would version datasets e.g., DVC or date-based storage and log the dataset path plus a file hash in MLflow so every model run can be reproduced later.